Matt Frisbie

Building Browser Extensions

Create Modern Extensions for Chrome, Safari, Firefox, and Edge

Foreword by Stefan Aleksic and Louis Vilgo, cofounders of Plasmo

Apress®

Matt Frisbie
California, CA, USA

ISBN 978-1-4842-8724-8 e-ISBN 978-1-4842-8725-5
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5

This Apress imprint is published by the registered company APress Media, LLC, part of Springer Nature.

The registered company address is: 1 New York Plaza, New York, NY 10004, U.S.A.

To my parents Mona and Patrick and my sister Lauren.You are all wonderful. I will continue to fill your bookshelves until you tell me to stop.

To Jordan. Your unwavering support advances the world of web development and keeps the corners of our carpets nice and flat.

Foreword

We are Stefan and Louis, co-founders of Plasmo. We first started our company building browser extensions for cyber security, experiencing firsthand the many difficulties extension developers face. We quickly realized that the process of building an extension was way too difficult. We decided to open source our framework, and dedicated our company's vision to solving the toughest problems in browser extension development. As of today, our framework is the most popular browser extension SDK worldwide, and our fast-growing extension product suite is used by some of the world's leading extension developers.

A large community soon gathered and shared even more issues blocking their development with us. We were excited to hear that Matt was writing a book on the subject! Extension development is tricky and filled with poor documentation, difficulty getting started, a massive transition from one extension version to another looming, and much more. There has never been a comprehensive guide to extension development, but Matt's book is changing that.

People use browsers for most of their lives. You might use your browser to read the news or find a good breakfast burrito recipe when you wake up. Software engineers might use their browsers to review pull requests and test their front end. Sales representatives use their browsers to send outbound messages to prospective clients on LinkedIn, and security engineers use their browsers to review new phishing alerts. These use cases are different, but each person uses the same tool to do their job.

Browser extensions augment the browser and transform it from a generic tool into a highly specialized one. A sales representative has different needs compared to a security engineer. In a few years, sales representatives and security engineers will have a stack of browser extensions that automate, categorize, and track their work. Some professions have picked up on this faster than others. Ask a high-performing sales representative about browser extensions they have installed, and you might hear a list of more than ten.

Browser extensions are potent but underutilized tools. The lack of know-how and a complete manual such as Matt's book has curbed people's imagination of what their browser could be. As knowledge of building extensions increases, there will be a tremendous appetite for new browser extensions that serve all kinds of niches we have never seen before.

This book encapsulates years of experience and research. It digs deep into critical details you could only find in random forums, obscure documentation, and fragmented source code online. It is everything you need to know about building browser extensions, from a well-rounded beginner-friendly introduction to a comprehensive guide for veterans. We especially love the following chapters:

- – Chapter **4**, "Browser Extension Architecture," is for adept developers who have yet to work with browser extensions.
- – Chapter **6**, "Understanding the Implications of Manifest V3," is for developers who need immediate help transitioning their extension to the newer manifest version.
- – Chapter **16**, "Tooling and Frameworks," is a must-read for developers who want to supercharge their workflow with robust frameworks and DevOps/CI automation to produce enterprise-grade browser extensions.

We can't wait to use your extension one day!

Stefan and Louis

Co-founders of Plasmo

Introduction

The world of browser extensions has far more than meets the eye. Consider the following:

- On average, 40% of Internet users in the United States use an ad-blocker on any device; overwhelmingly, these adblockers take the form of browser extensions.
- The tech company Honey, whose primary product is a browser extension, was acquired by PayPal in 2020 for $4 billion.
- As of 2021, there were 1.8 million apps in Apple's App Store; the Chrome Web Store has 180,000 extensions.

When I saw there were 0 relevant Amazon search results for "build chrome extension," I nearly fell out of my chair. I knew at once that this book *must* be written.

Building Browser Extensions: Create Modern Extensions for Chrome, Safari, Firefox, and Edge covers all the knowledge you will need to write cross-browser extensions with the latest web development tools. Browser extensions are given access to *extremely* powerful APIs. I believe most developers are blind to that power – and unaware of just how much it is within their reach.

This book is designed to enlighten web developers and illuminate the true potential of the browser extension software platform. It is geared for developers who have experience building websites and can apply their knowledge to a new software domain. This book is not ideal for people new to programming – it would be like an inexperienced cook starting off by learning to make a sauce.

A major barrier to developing browser extensions is the appalling status quo of documentation. The fragmentation between different browsers and different manifest versions turns slogging through the documentation into a mind-numbingly onerous affair. I wrote this book specifically to address this problem. The reader will learn what is possible with the APIs, how they can best be applied, and all the traps to avoid. The book is not intended to *replace* the API documentation, as it is changing all the time. Instead, it is intended to supplement the API documentation; the book has plenty of direct links to the Chrome Developers and MDN sites throughout.

The transition to manifest v3 is upon us, and already it is causing problems. If you are confused about what manifest v3 is, what are its implications, and how best to navigate the ongoing transition, this book is for you. I dedicated an entire chapter to the manifest v2/v3 transition.

The lingua franca of web development is React, and this book gives special attention to the best ways in which you can write a browser extension in React. It also covers all the supplemental tools you'll need along the way, such as Webpack, Parcel, and Plasmo.

Like many developers, I learn by example. I was annoyed that so many APIs listed in the documentation were totally inscrutable. For example, the omnibox API is amazing and incredibly useful, but the documentation on how to use it is *garbage*. I just wanted a simple example to pick apart and play with, and there was nothing to be found. To fill this need, I created a companion extension for the book: *Browser Extension Explorer*. It's an open source browser extension with dozens of interactive demos. Each demo shows how various browser extension pieces and APIs work, and each includes links to the specific source files so you can see how it was built.

You can download *Browser Extension Explorer* from the Chrome Web Store, or find a link to it on the companion website to this book: `https://buildingbrowserextensions.com`.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub via the book's product page, located at https://github.com/Apress/Building-Browser-Extensions-by-Matt-Frisbie. For more detailed information, please visit http://www.apress.com/source-code.

Acknowledgments

I'd like to acknowledge the tremendous work done by editors Divya Modi, Shonmirin P.A., James Markham, and everyone else involved with the book's production. A capable and agile staff does so much to improve the quality of the product. You were all invaluable resources, and it was a pleasure working with you.

I'd also like to thank the book's technical reviewer Jeff Friesen. Technical authors are nothing without this meticulous work behind the scenes, and his contributions were nothing short of outstanding.

Finally, I would like to thank Apress for publishing this book with me. This was my first title where the book was entirely my own concept, and I am thankful that it found a home with such a terrific publisher.

Table of Contents

Chapter 1: What Are Browser Extensions?
History of Browser Extensions
Customizing Software with Plugins
History of Web Browsers
Native Browser Plugins
From Browser Add-ons to Extensions
The Browser Extension Landscape
Comparing Mobile Apps and Browser Extensions
Browser Extension Stores
Types of Browser Extensions
Summary
Credits
Chapter 2: Fundamental Elements of Browser Extensions
The Browser Model
Browser Tabs
Same-Origin Policy
The Browser Extension Model
Independent JavaScript Pages and Runtimes
Native APIs and User Interfaces
Tab and Domain Access
Observing and Intercepting Network Requests
Elements of Browser Extensions
Extension Manifest
Background Scripts
The Popup Page
The Options Page
Content Scripts
Devtools Panels and Sidebars
Extension Elements in Action
Honey
LastPass
Grammarly
React Developer Tools
Summary
Chapter 3: Browser Extension Crash Course
Creating the Manifest
Minimum Viable Extension
Installing Your Extension
Reloading Your Extension

Adding a Background Script
Adding a Popup Page
Adding an Options Page
Adding a Content Script
Working with Multiple Tabs
Adding a Devtools Panel
Summary
Chapter 4: Browser Extension Architecture
Architecture Overview
Plurality, Lifecycles, and Updates
Background Service Worker
Popup and Options Pages
Devtools Pages
Content Scripts
Browser Extension File Server
Sandboxed Pages
Summary
Chapter 5: The Extension Manifest
The Manifest File
Supporting Different Locales
Match Patterns and Globs
File Path Match Patterns
URL Match Patterns
URL Globs
Manifest Properties
action
author
automation
background
browser_action
browser_specific_settings
chrome_settings_overrides
chrome_url_overrides
commands
content_capabilities
content_scripts
content_security_policy
converted_from_user_script
cross_origin_embedder_policy
cross_origin_opener_policy

declarative_net_request
default_locale
description
developer
devtools_page
differential_fingerprint
event_rules
externally_connectable
file_browser_handlers
file_system_provider_capabilities
homepage_url
host_permissions
icons
incognito
key
manifest_version
minimum_chrome_version
nacl_modules
name
oauth2
offline_enabled
omnibox
optional_host_permissions
optional_permissions
options_page
options_ui
page_action
permissions
platforms
replacement_web_app
requirements
sandbox
short_name
storage
system_indicator
tts_engine
update_url
version
version_name
web_accessible_resources

[Summary]
[Chapter 6: Understanding the Implications of Manifest V3]
[Motivation for Manifest V3]
[Security]
[Performance]
[Privacy and Transparency]
[Revenue]
[Implications of Background Service Workers]
[DOM]
[XMLHttpRequest]
[Timer API]
[Event Handlers]
[Service Worker Persistence]
[Global State and Storage]
[Audio and Video]
[Implications of Content Security Policy Restrictions]
[Implications of DeclarativeNetRequest]
[Summary]
[Chapter 7: Background Scripts]
[Web Page Service Workers vs. Extension Service Workers]
[Similarities]
[Differences]
[Manifest v2 vs. Manifest v3]
[Scripts vs. Service Workers]
[JavaScript Imports]
[No Access to DOM and Limited Global APIs]
[Nonpersistent]
[No Shutdown Event]
[No Programmatic Background Access]
[Working with Background Scripts]
[Inspecting Background Service Workers]
[Service Worker Errors]
[Service Worker Termination]
[Common Patterns]
[Event Handler]
[Secret Management and Authentication]
[Message Hub]
[Storage Manager]
[Injecting Scripts]
[Sniffing Web Traffic]

Installed/Updated Events
Opening Tabs
Forcing Service Worker Persistence
Implementation
Drawbacks and Limitations
Summary
Chapter 8: Popup and Options Pages
Popup Pages
Popup Page Properties
Opening and Closing Popup Pages
Changing the Popup Page
Detecting Popup State
Suggested Use
Options Pages
Options Page Properties
Opening and Closing Options Pages
Detecting Options Pages
Suggested Use
Content Script Restrictions
Summary
Chapter 9: Content Scripts
Introduction to Content Scripts
Injecting CSS
Content Script Isolation
Page Automation
Logging and Errors
Extension API Access
Modules and Code Splitting
Bundling
Dynamic Imports
Dynamic Script Tags
Specialized Content Script Properties
Programmatic Injection
Summary
Chapter 10: Devtools Pages
Introduction to Devtools Pages
Creating a devtools Page
Adding panels and sidebars
The Devtools API
Sniffing network traffic

[Inspecting the Page]
[Content Scripts and Devtools Messaging]
[Other Devtools API Features]
[Summary]
[Chapter 11: Extension and Browser APIs]
[Global API Namespace]
[Promises vs. Callbacks]
[Error Handling]
[Context-restricted APIs]
[Events API]
[Format]
[Event Filtering]
[WebExtensions API Quick Reference]
[Permissions]
[Messaging]
[Storage]
[Authentication]
[Network Requests]
[Internationalization]
[Browser and System Control]
[Page and Screen Capture]
[Proxy]
[Browser State Management]
[Tabs and Windows]
[Debugger]
[Search]
[Alarms]
[Scripting]
[DOM]
[Text to Speech]
[Privacy]
[Idle]
[Devtools]
[Extension Introspection]
[Extension Management]
[System State]
[Enterprise Only]
[Firefox Only]
[Chrome OS Only]
[Deprecated]

Summary
Chapter 12: Permissions
Permissions Basics
Checking Permissions
Using Optional Permissions
Granting Permissions Declaratively vs. Imperatively
Permission Request Idempotence
Host Permissions
Permissions Lifetime
Permissions Warnings
Testing Permissions Warnings
Considerations for Published Extensions
Triggering the Slow Review Queue
Auto-Disable Updates
Permissions List
activeTab
alarms
background
bookmarks
browserSettings
browsingData
captivePortal
certificateProvider
clipboardRead
clipboardWrite
contentSettings
contextMenus
contextualIdentities
cookies
debugger
declarativeContent
declarativeNetRequest
declarativeNetRequestFeedback
declarativeWebRequest
desktopCapture
Devtools page
dns
documentScan
downloads
downloads.open

enterprise.deviceAttributes
enterprise.hardwarePlatform
enterprise.networkingAttributes
enterprise.platformKeys
experimental
fileBrowserHandler
fileSystemProvider
find
fontSettings
gcm
geolocation
history
Host Permissions
identity
idle
loginState
management
menus
menus.overrideContext
nativeMessaging
notifications
pageCapture
pkcs11
platformKeys
power
printerProvider
printing
printingMetrics
privacy
processes
proxy
scripting
search
sessions
signedInDevices
storage
system.cpu
system.display
system.memory
system.storage

tabCapture
tabGroups
tabHide
tabs
theme
topSites
tts
ttsEngine
unlimitedStorage
vpnProvider
wallpaper
webNavigation
webRequest
webRequestBlocking
Summary
Chapter 13: Networking
Comparing Websites and Extensions
Networking Architecture
Options Pages
Popup and Devtools Pages
Content Scripts
Background Scripts
Pinning an Extension ID
Authentication Styles
No Authentication
Content Script Spoofing
Cookie Authentication
Json Web Token Authentication
OAuth and OpenID
OAuth, OpenID, and the Identity API
OAuth API Methods
OAuth Redirect URLs
Configuring the Authorization Platform
Additional Help
OAuth and OpenID Examples
Google OAuth with getAuthToken()
Google OpenID with launchWebAuthFlow( )
Manual Github OAuth with launchWebAuthFlow()
Networking APIs
The webNavigation API

[The webRequest API]
[The declarativeNetRequest API]
[Summary]
[Chapter 14: Extension Development and Deployment]
[Local Development]
[Inspecting Your Extension]
[File Changes]
[Error Monitoring]
[Extension Reloads]
[Automated Extension Tests]
[Unit Tests]
[Integration Tests]
[Additional Reading]
[Publishing Extensions]
[Store Listing]
[Privacy Practices]
[Review Process]
[Updating Extensions]
[Update Considerations]
[Cancelling Updates]
[Automated Update Publishing]
[Tracking User Activity]
[Dashboard Metrics]
[Analytics Libraries]
[Setting Up Google Analytics]
[Install and Uninstall Events]
[Summary]
[Chapter 15: Cross-Browser Extensions]
[Introduction to Cross-Browser Support]
[Browser Coverage Tradeoffs]
[Browser Share]
[Chromium Browser Extension Sharing]
[Adapting Your Codebase]
[API Probing]
[Differential Manifests]
[Manifest v2/v3]
[Extension Marketplaces]
[Marketplace Similarities]
[Marketplace Differences]
[Chrome Web Store]

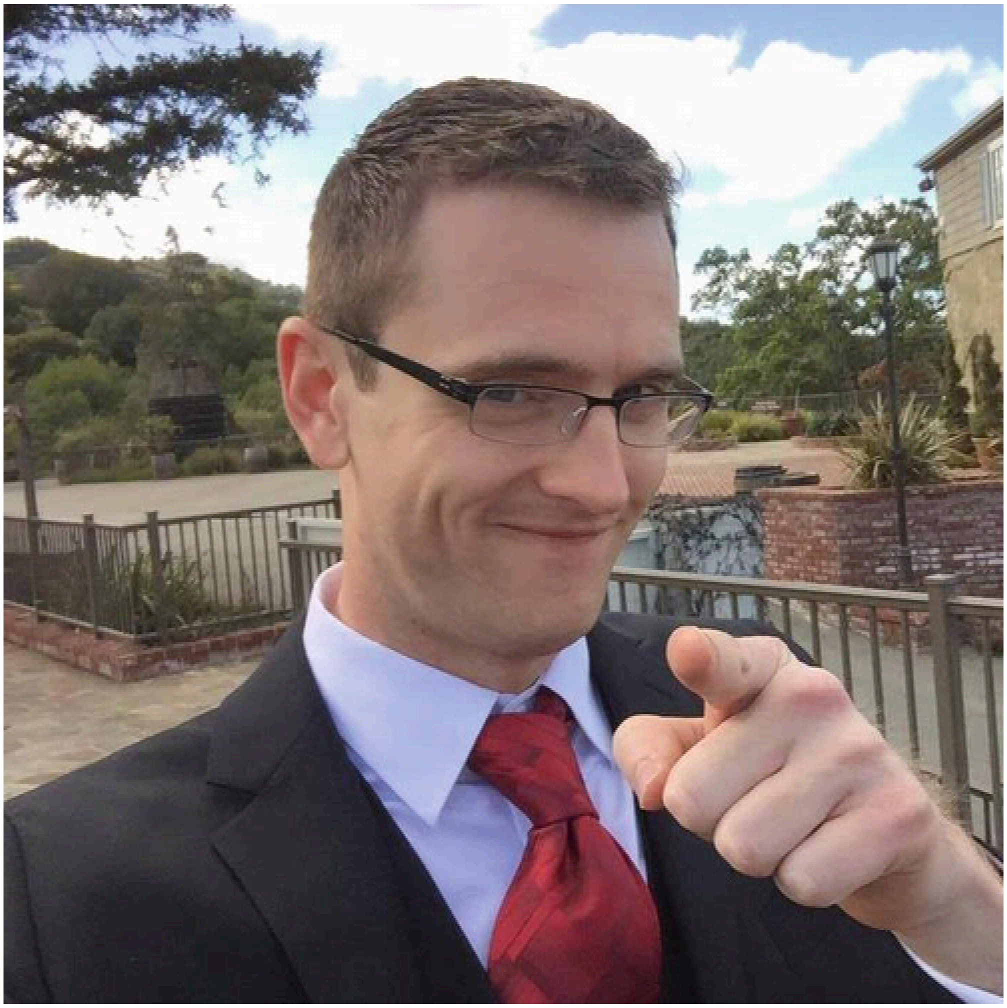
Add-ons for Firefox
Microsoft Edge Add-ons
Safari Extensions App Store
Opera Addons
Mobile Extensions
Mobile Extension User Interfaces
Kiwi Browser
Firefox Mobile
iOS Safari
Automated Deployment
WebExtensions API Support
Safari Extension Development
Prerequisites
Architecture
Creating an Extension Project
Writing the App
Testing on macOS
Testing on iOS
Deploying to the App Store
Converting an Existing Extension
Firefox Idiosyncrasies
Manifest Versions
Sidebars
API Additions
Summary
Chapter 16: Tooling and Frameworks
Building Extensions with React
Single Entrypoint React Extensions
Multiple Entrypoint React Extensions
Reactive State Management
Routing
Mozilla Tools
web-ext
webextension-polyfill
Bundlers and CLI Tools
Parcel
Webpack
Plasmo
High-level Overview
JavaScript Frameworks

Documentation and Examples
Differential Build Outputs
Automatic Manifest Generation
Icon Generation
Bundling Remote Code
Environment Variables
Content Script Mounting
Extension-friendly Hot Module Replacement
Automated Deployment with Browser Platform Publish
Useful Sites
Summary
Index

About the Author

Matt Frisbie

has



worked in web development for over a decade. During that time, he's been a startup co-founder, an engineer at a Big Four tech company, and the first engineer at a Y Combinator startup that would eventually become a billion-dollar company. As a Google software engineer, Matt worked on both the AdSense and Accelerated Mobile Pages (AMP) platforms; his code contributions run on most of the planet's web browsing devices. Prior to this, Matt was the first engineer at DoorDash, where he helped lay the foundation for a company that has become the leader in online food delivery. Matt has written three books, *Professional JavaScript for Web Developers*, *Angular 2 Cookbook*, and *AngularJS Web Application Development Cookbook*, and recorded two video series, "Introduction to Modern Client-Side Programming" and "Learning AngularJS." He speaks at frontend meetups and webcasts, and is a level 1 sommelier. He majored in Computer Engineering at the University of Illinois Urbana-Champaign. Matt's Twitter handle is @mattfriz.

About the Technical Reviewer

Jeff Friesen

is a

freelance software developer and educator conversant in multiple operating systems, programming languages, and numerous technologies. He is currently exploring bare metal programming for the Raspberry Pi and developing his own assembler and associating tooling to facilitate the development of a simple Pi-based operating system.